

Using AI as a Companion

To Start...

- Technology is evolving fast!
- Constantly – improving but overwhelming
- The hype is real, 90% of companies have adopted AI co-creation
 - > 80% says it has increased productivity
 - 30% have little trust in it.



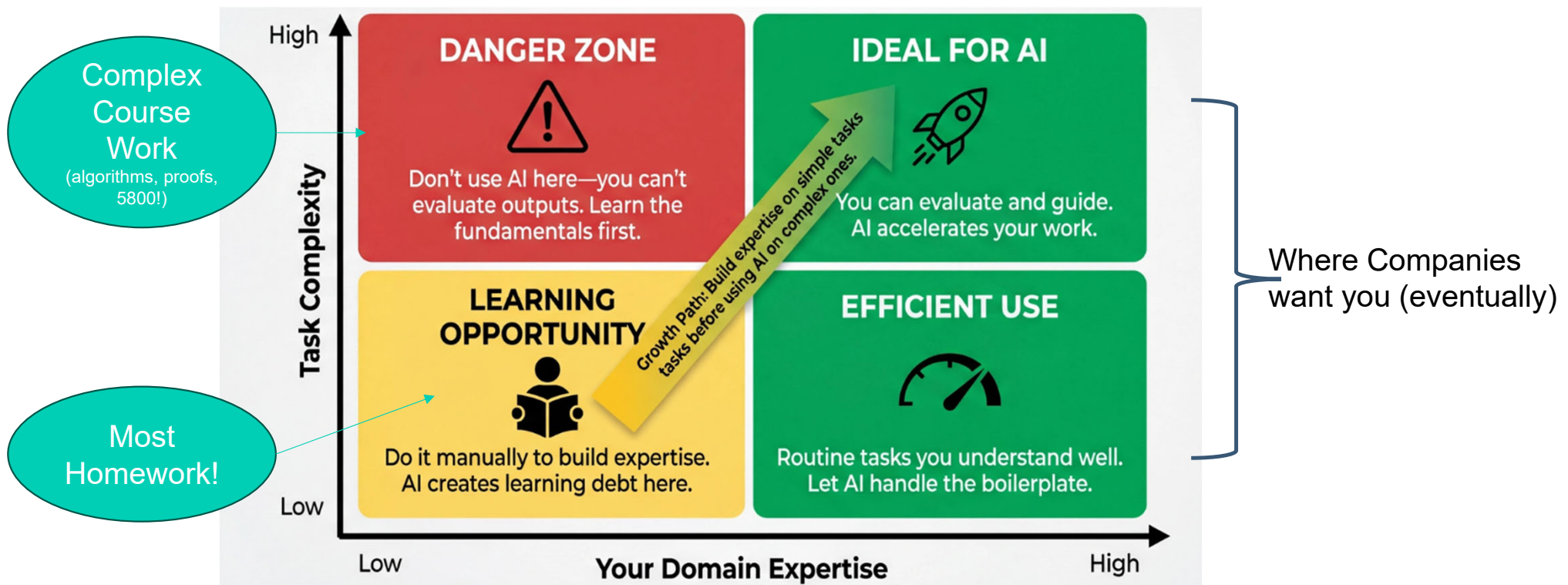
DeBellis, Derek, Kevin M. Storer, and Nathen Harvey. *State of AI-Assisted Software Development 2025*. DORA/Google Cloud, 2025.

Northeastern University

Concerns are real

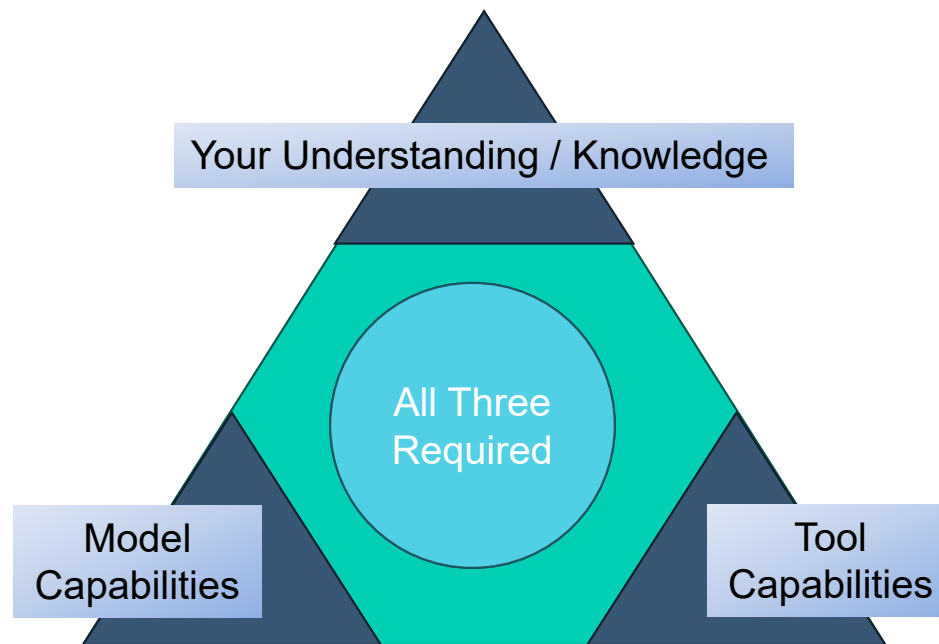
- Sustainability / Environment Impact
- Cognitive Dept
- Code Instability
- Lack of Oversight
- Security Concerns

Fundamental



Bell, Jonathan. "Lecture 13: AI Coding Assistants." CS 3100: Program Design & Implementation II, Northeastern University, 2026. <https://neu-pdi.github.io/cs3100-public-resources/lecture-slides/l13-ai-coding-assistants#/23>.

Human Element – Still Dominate



Co-creation Workflow



Context is Everything!

SE Principles Matter More!

Bell, Jonathan. "Lecture 13: AI Coding Assistants." CS 3100: Program Design & Implementation II, Northeastern University, 2026. <https://neu-pdi.github.io/cs3100-public-resources/lecture-slides/113-ai-coding-assistants/#/15>.

Based on [research on Developer-AI collaboration](#) from Google

Co-creation projects == more setup

Good SE practices these are the norm

- Version Control – git
 - Include pre-commit hooks! (auto checks on commit)
- Linters
- Formatters
- Type Checks
- Complexity Review
- Dependency Reviews

Don't Forget SE Tools

Category	Why It Matters	Python	JavaScript / TS	Java	Rust
 Lint	Catches style & logic errors before runtime — no debates	ruff or flake8	eslint	Checkstyle PMD	cargo clippy (built-in)
 Formatter	Formatting is never a human decision	ruff or black	prettier	google-java-format	cargo rustfmt (built-in)
 Type Checker	Catches type mismatches without running code	mypy or pyright	tsc (compiler)	Compiler	Compiler
 Complexity Analyzer	High cyclomatic complexity → hard to test or review	ruff or radon	eslint complexity rules	PMD SonarQube	cargo clippy
 Dependency Scanner	Known vulnerabilities in third-party packages	pip-audit or safety	npm audit snyk	OWASP Dep-Check	cargo audit

Version Control & Pre-Commit Hooks

Why Version Control?



Full history

Every change is tracked — revert anything, anytime



Safe experimentation

Branches let you try ideas without breaking working code



Collaboration

Multiple people work on the same codebase without conflict

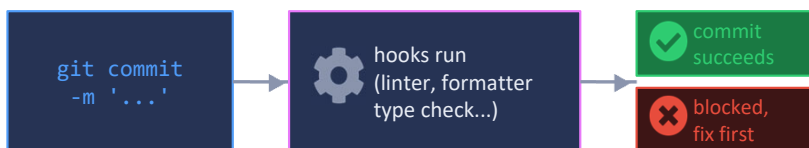


Accountability

Who changed what, and why — essential for code review

Pre-Commit Hooks: The Enforcement Layer

Run your quality tools automatically on every commit — before code enters the repo



```
.pre-commit-config.yaml

repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    hooks: [{ id: ruff }]
  - repo: https://github.com/psf/black
    hooks: [{ id: black }]
  - repo: https://github.com/pre-commit/mirrors-mypy
    hooks: [{ id: mypy }]
```

Key insight: Pre-commit hooks turn the quality tools from the previous slide into **mandatory gates** — no one can forget to run them, including AI-generated code.

Adding in tools, Process is

- Iterative Design
- For each feature
 - Carefully define requirements – how do you measure success?
 - Branch - repeat each step as needed
 - Identify
 - Engage
 - Evaluate
 - Calibrate
 - Tweak
 - Finalize
 - Test, Update Documentation, Confirm Standards
 - Code audit (security and quality)
- Merge and Repeat
 - Working in teams? Merge is reviewed by another team member!

With all that said

- Let's start building together

Employer Partner Day